

Lab 4 – Solutions

October 14, 2024

This lab deals with the numerical resolution of a linear system

$$\mathbb{A}\mathbf{x} = \mathbf{b}, \quad \text{for } \mathbb{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n.$$

Today we will focus on direct methods (i.e. methods for which the solution is obtained in a **finite number** of operations).

1 Solution of triangular systems

Consider a generic non singular lower triangular system

$$\mathbb{L}\mathbf{x} = \mathbf{b}$$

In order to have $\mathbb{L}$ non singular, its diagonal elements must be non zero. The system reads;

$$\begin{bmatrix} l_{1,1} & 0 & \cdots & 0 \\ l_{2,1} & l_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n,1} & l_{n,2} & \cdots & l_{n,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

Forward substitution

$$x_1 = \frac{b_1}{l_{11}},$$
$$x_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij}x_j \right), \quad i = 2, \dots, n.$$

Comments

Computational cost: $\mathcal{O}(n^2)$.

Similarly, consider a generic non singular upper triangular system

$$\mathbb{U}\mathbf{x} = \mathbf{b}$$

Again, in order to have $\mathbb{U}$ non singular, its diagonal elements must be non zero. The system reads;

$$\begin{bmatrix} u_{1,1} & u_{1,2} & \cdots & u_{1,n} \\ 0 & u_{2,2} & \cdots & u_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{n,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

Backward substitution

$$x_n = \frac{b_n}{u_{nn}},$$

$$x_i = \frac{1}{u_{ii}} \left(b_i - \sum_{j=i+1}^n u_{ij}x_j \right), \quad i = n-1, \dots, 1.$$

Comments

Computational cost: $\mathcal{O}(n^2)$.

Exercise 4.1. a. Write a function which implements the backward substitution method for solving a generic upper triangular system. Apply such a function to solve the linear system $\mathbb{A}\mathbf{x} = \mathbf{b}$ with

$$\mathbb{A} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

b. Write a function to implement the forward substitution method to solve lower triangular systems. Apply this function to the system $\mathbb{A}^T\mathbf{x} = \mathbf{b}$. with $\mathbb{A}$ and $\mathbf{b}$ defined as above.

c. Use the backward and the forward algorithms to solve the system $\mathbb{A}^T\mathbb{A}\mathbf{x} = \mathbf{b}$.

Solution Exercise 4.1.

backward_substitution.m

```
function x = backward_substitution(A,b)
%BACKWARD_SUBSTITUTION Solve an upper triangular system using backward
%substitution method.
% x = backward_substitution(A,b)
%
% Inputs : A = system coefficient matrix
%          b = right-hand side vector
% Outputs : x = solution vector

n = numel(b);
x = zeros(size(b));
x(n) = b(n) / A(n,n);

for (k = n-1:-1:1)
    x(k) = (b(k) - A(k,k+1:n)*x(k+1:n)) / A(k,k);
end

end
```

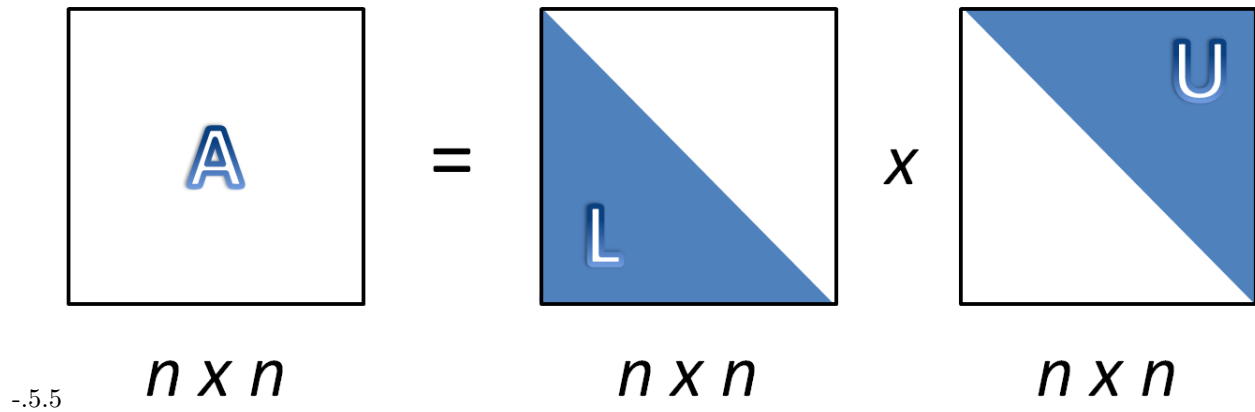
forward_substitution.m

```
function x = forward_substitution(A,b)
%FORWARD_SUBSTITUTION Solve an lower triangular system using forward
%substitution method.
% x = forward_substitution(A,b)
%
% Inputs : A = system coefficient matrix
%          b = right-hand side vector
% Outputs : x = solution vector

n = numel(b);
x = zeros(size(b));
x(1) = b(1) / A(1,1);

for ii=2:n
    x(ii) = (b(ii) - A(ii, 1:ii-1) * x(1:ii-1)) / A(ii,ii);
end

end
```



ex_4_1.m

```

%% Exercise 4.1

% Write a function which implements the backward substitution
% method for solving a generic upper triangular system. Apply
% such a function to solve the linear system  $Ax = b$  with

%A = [1 2 3 4;
%     0 1 2 3;
%     0 0 1 2;
%     0 0 0 1];
%b = [1 1 1 1]';

A=diag(ones(4,1)) + 2 * diag(ones(3,1),1) + 3 * diag(ones(2,1),2) ;
A(1, end)=4;

b= ones(4,1);

x_bwd = backward_substitution(A, b)

% Check with builtin \ operator of MATLAB
x_mat=A \ b

% Write a function which implements the backward substitution
% method for solving a generic upper triangular system. Apply
% such a function to solve the linear system  $A'x = b$  with

x_fwd = forward_substitution(A', b)

% Check with builtin \ operator of MATLAB
x_mat2=A' \ b

% Use the backward and the forward algorithms to solve the
% system A
y = forward_substitution(A', b);
x = backward_substitution(A, y)

% Check with builtin \ operator of MATLAB
(A'*A) \ b

```

2 LU decomposition

Let $A \in \mathbb{R}^{n \times n}$: the LU decomposition of A allows to write

$$A = LU$$

with L **lower triangular** matrix s.t. $l_{ii} = 1, \forall i = 1 \dots n$, and U **upper triangular** matrix.

The existence and the uniqueness of such decomposition is related to the principal submatrices of order i of the matrix $\mathbb{A}$, with $i = 1, \dots, n-1$, which are demanded to be non singular (necessary and sufficient condition). As an alternative, the sufficient condition holds:

- $\mathbb{A}$ is a strictly **diagonally dominant** by row $|a_{ii}| \geq \sum_{j=1, j \neq i}^n |a_{ij}| \forall i$ OR by column matrix
- $\mathbb{A}$ is a **symmetric positive definite** matrix. $\mathbb{A} = \mathbb{A}^T$ and $\mathbf{x}^T \mathbb{A} \mathbf{x} > 0 \quad \forall \mathbf{x} \neq \mathbf{0}$.

LU decomposition in MATLAB/Octave

```
>> help lu
lu    lu factorization
[L,U,P] = lu(A)
returns unit lower triangular matrix L, upper triangular matrix U, and permutation
matrix P so that P*A = L*U.

[L,U] = lu(A)
stores an upper triangular matrix in U and a "psychologically lower triangular matrix
" (i.e. the product of a lower triangular and a permutation matrix) in L, so that A =
L*U, A can be rectangular.
```

LU decomposition with no pivoting

$$\left. \begin{array}{l} \mathbb{A}\mathbf{x} = \mathbf{b} \\ \mathbb{A} = \mathbb{L}\mathbb{U} \end{array} \right\} \longrightarrow \mathbb{L}\mathbb{U}\mathbf{x} = \mathbf{b} \longrightarrow \left\{ \begin{array}{l} \mathbb{L}\mathbf{y} = \mathbf{b} \\ \mathbb{U}\mathbf{x} = \mathbf{y} \end{array} \right.$$

When the standard LU factorization fails, dealing with non-singular matrices, we can resort to the **LU factorization with pivoting**. Pivoting is a technique which changes the order of the rows of a matrix in order to avoid null pivot elements during the LU factorization.

LU decomposition with pivoting

$$\left. \begin{array}{l} \mathbb{A}\mathbf{x} = \mathbf{b} \\ \mathbb{P}\mathbb{A} = \mathbb{L}\mathbb{U} \end{array} \right\} \longrightarrow \mathbb{P}\mathbb{A}\mathbf{x} = \mathbf{b} \longrightarrow \left\{ \begin{array}{l} \mathbb{L}\mathbf{y} = \mathbb{P}\mathbf{b} \\ \mathbb{U}\mathbf{x} = \mathbf{y} \end{array} \right.$$

Notice that the right-hand side of the lower triangular system is modified with respect to the standard LU factorization

This approach is ideal to solve multiple systems $\mathbb{A}\mathbf{x} = \mathbf{b}_i$ with different right hand-sides $\mathbf{b}_i$, and all sharing the same coefficient matrix. Actually it suffices to compute the decomposition $\mathbb{L}\mathbb{U}$ just **once**.

Exercise 4.2. Consider the system $\mathbb{A}\mathbf{x} = \mathbf{b}$ with

$$\mathbb{A} = \begin{bmatrix} 2 & 10 & 4 & 0 \\ 1 & 0 & 2 & 2 \\ 1 & 4 & 0 & 2 \\ 1 & 2 & 1 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 10 \\ 1 \\ 3 \\ 3 \end{bmatrix}.$$

- Compute the solution of the system $\mathbb{A}\mathbf{x} = \mathbf{b}$ using the $\mathbb{L}\mathbb{U}$ decomposition of matrix $\mathbb{A}$.
- Find the determinant of $\mathbb{A}$ without using the command `det`.

Solution Exercise 4.2.

ex_4_2.m

```
%% Exercise 4.2
clc
clear all
close all
%% Compute the solution of the system Ax = b using the LU decomposition of matrix A
A = [2 10 4 0;
```

```

    1 0 2 2;
    1 4 0 2;
    1 2 1 1;
b = [10 1 3 3]';

[L, U] = lu(A);
% In this case it is not essential to include the matrix P among the output
% variables in order to have L in the form of a lower triangular matrix
% with L_ii = 1 for each i.

y=forward_substitution(L, b)
x=backward_substitution(U,y)

% Check
y = L \ b; %forward
x = U \ y %backward
% Check by solving the sysyem directly
A \ b

% Thanks to the Binet-Cauchy formula, we have, for generic square matrices,
% det(A*B) = det(A)*det(B).
% In our case, since A = L*U, det(A) = det(L*U) = det(L)*det(U).
% However L_ii = 1 for each i, so det(L) = 1
% => det(A) = det(U).

%% Find the determinant of A without using the command det.
detA = prod(diag(U))

% Check with the command det
det(A)

```

Exercise 4.3. Let us consider the matrix $\mathbb{A} = \begin{bmatrix} 50 & 1 & 3 \\ 1 & 6 & 0 \\ 3 & 0 & 1 \end{bmatrix}$, $\mathbb{B} = \begin{bmatrix} 4 & 1 & 1 \\ 4 & 1 & 2 \\ 1 & 0 & 15 \end{bmatrix}$.

1. Apply the Matlab[®] command `lu` and compute the LU factorization of the matrices $\mathbb{A}$, $\mathbb{B}$.
2. Compute the determinant of the $\mathbb{A}, \mathbb{B}$ without the command `det` if possible.
3. Solve the systems $\mathbb{A}\mathbf{x} = \mathbf{a}, \mathbb{B}\mathbf{x} = \mathbf{b}$. Choose the vectors $\mathbf{a}, \mathbf{b}$, such that the solution of the system is $\mathbf{x}_{ex} = [1, 1, 1]^T$.
4. Compute the solution of the system $\mathbb{A}\mathbf{x} = \mathbf{b}$, by using the backward and forward substitution the functions previously implemented.

Solution Exercise 4.3.

ex_4_3.m

```

%% Exercice 4.3
clc
clear all
close all
%% Apply the Matlab command lu and compute the LU factorization of the matrices
A = [50 1 3;
     1 6 0;
     3 0 1];
B = [4 1 1;
     4 1 2;
     1 0 15];

[La, Ua] = lu(A);
[Lb, Ub] = lu(B);
% Let us check if the matrices L and U are as we expect them to be.

La

```

```

Ua
Lb
Ub

% The matrix B does not admit a unique LU factorization because there is a singular
% principal submatrix. Pivoting is required to restore the uniqueness of the factorization
[La, Ua, Pa] = lu(A);
[Lb, Ub, Pb] = lu(B);

% Pa is the identity matrix as expected. Pb is different from I.
Pa
Pb

% Compute the determinant of the A,B without the command det
% in the case of matrix A, let procede as in ex 4.2
detA= prod(diag(Ua))
%check
detA_mat=det(A)
% in the case of B, notice that P could have determinanti =+-1, therefore
% det(P)det(B)=det(Ub)---> det(B)= (det(Ub)/det(P))
detb=prod(diag(Ub))/det(Pb)
detV_mat=det(B)
%% Solve the systems Ax = a, Bx = b. Choose the vectors a, b, such that
% the solution of the system is xex = [1, 1, 1]'
xex = [1; 1; 1];
b = A*xex;

y = forward_substitution(La, b);
x = backward_substitution(Ua, y);

% Finally, let's check if x is equal to xex:

x

% Now with matrix B
b = B*xex;

y = forward_substitution(Lb, Pb*b);
x = backward_substitution(Ub, y);
x

```

Exercise 4.4 (LU decomposition). Consider the system $\mathbb{A}\mathbf{x} = \mathbf{b}$ with

$$\mathbb{A} = \begin{bmatrix} 12 & 5 & -8 & -5 \\ -4 & -4 & 8 & -6 \\ 4 & 2 & -3 & 0 \\ 0 & -1 & 2 & -4 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 4 \\ -6 \\ 3 \\ -3 \end{bmatrix}.$$

- Use the LU decomposition of matrix $\mathbb{A}$ to solve the system. Use the `lu` MATLAB/Octave command.
- Compute the determinant of $\mathbb{A}$ using the LU decomposition.
- Consider now a different right-hand side vector $\mathbf{c} = [-22, -12, -1, -11]^T$. Use an approach which allow us to contain the computational cost.

Solution Exercise 4.4.

ex_4_4.m

```
%% Homework 4.1
clc
clear all
close all
%% Use the LU decomposition of matrix A to solve the system. Use both
% the functions implemented and the lu MATLAB/Octave command.
A = [12 5 -8 -5;
     -4 -4 8 -6;
      4 2 -3 0;
      0 -1 2 -4;];

b = [4 -6 3 -3]';

[L, U] = lu(A)

y1 = forward_substitution(L, b);
x1 = backward_substitution(U, y1)

x1_mat=A\b
% Compute the determinant of A using the LU decomposition
detA = prod(diag(U))
det(A)

c = [-22 -12 -1 -11]';

y2 = forward_substitution(L, c);
x2 = backward_substitution(U, y2)

x2_mat=A\c

% The solution of the system A*x2 = c is obtained without a new
% factorization, since we stored the matrices L and U the first time we
% solved the system with b. This factorization is indeed useful everytime
% the matrix A does not change and we have to solve several linear systems
% with different right-hand sides.
```

Inverse Matrix

We can define the inverse of a squared matrix $\mathbb{A} \in \mathbb{R}^{n \times n}$ as the matrix $\mathbb{X} = \mathbb{A}^{-1} \in \mathbb{R}^{n \times n}$ such that $\mathbb{A}\mathbb{X} = \mathbb{X}\mathbb{A} = \mathbb{I}$. It is possible to determine $\mathbb{A}^{-1}$ by solving the following n linear systems:

$$\mathbb{A}\mathbf{v}_i = \mathbf{e}_i, \quad i = 1, \dots, n,$$

where $\mathbf{e}_i$ denote the consecutive columns of the matrix $\mathbb{I}$ (i.e the vectors of the standard basis of $\mathbb{R}^n$). Thus it turns out that

$$\mathbb{A}^{-1} = [\mathbf{v}_1 | \mathbf{v}_2 | \dots | \mathbf{v}_n]$$

Exercise 4.5. 1. Write a function `[InvA]=MyInv(A)` that computes the inverse `InvA` of a generic square matrix `A`.

2. Use the function `MyInv` to compute the inverse of the matrix `A` in the previous exercise. Compare the result with the output provided by Matlab with command `inv`.

Solution Exercise 4.5.

ex_4_4.m

```

%% Homework 4.1
clc
clear all
close all
%% Use the LU decomposition of matrix A to solve the system. Use both
% the functions implemented and the lu MATLAB/Octave command.
A = [12 5 -8 -5;
     -4 -4 8 -6;
       4 2 -3 0;
       0 -1 2 -4];

b = [4 -6 3 -3]';

[L, U] = lu(A)

y1 = forward_substitution(L, b);
x1 = backward_substitution(U, y1)

x1_mat=A\b
% Compute the determinant of A using the LU decomposition
detA = prod(diag(U))
det(A)

c = [-22 -12 -1 -11]';

y2 = forward_substitution(L, c);
x2 = backward_substitution(U, y2)

x2_mat=A\c

% The solution of the system A*x2 = c is obtained without a new
% factorization, since we stored the matrices L and U the first time we
% solved the system with b. This factorization is indeed useful everytime
% the matrix A does not change and we have to solve several linear systems
% with different right-hand sides.

```

MyInv.m

```

function [InvA] = MyInv(A)

[L, U, P] = lu(A);
%s = size(A);
%n = s(1);
n = length(A);
InvA=zeros(size(A));
for k = 1:n
    e = zeros(n,1);
    e(k) = 1;
    y = forward_substitution(L, P*e);
    InvA(:,k) = backward_substitution(U,y);
end
end

```

3 Cholesky decomposition

Let $A \in \mathbb{R}^{n \times n}$ be a **symmetric** and **positive definite** matrix: then there $\exists!$ **upper triangular**, with $h_{ii} > 0$, s.t.

$$A = H^T H.$$

For any symmetric matrix A A pos. def. $\iff$

- $x^T A x > 0, \quad \forall x \neq 0$
- $\lambda_i > 0, \quad \forall i = 1, \dots, n, \quad \lambda_i$ eigenvalue of A

- $\det(\mathbb{A}_i) > 0, \quad \forall i = 1, \dots, n$

$\mathbb{A}$ pos. def. $\iff$

- $\mathbb{A}$ is diagonally dominant, with all the diagonal elements being positive.

The elements of $\mathbb{H}$ can be computed by the following algorithm:

Cholesky decomposition

We set $h_{11} = \sqrt{a_{11}}$ and for $i = 2, \dots, n$,

$$h_{ij} = \frac{1}{h_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} h_{ik} h_{jk} \right) \quad j = 1, \dots, i-1$$

$$h_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} h_{ik}^2}$$

Cholesky factorization is available in MATLAB via the command $\mathbf{R} = \text{chol}(\mathbf{A})$, where $\mathbf{R}$ is the upper triangular matrix $\mathbb{H}$.

Cholesky decomposition in MATLAB

```
>> help chol
CHOL    Cholesky factorization.
CHOL(A) uses only the diagonal and upper triangle of A. The lower
triangle is assumed to be the (complex conjugate) transpose of the
upper triangle. If A is positive definite, then R = CHOL(A)
produces an upper triangular R so that R'*R = A. If A is not
positive definite, an error message is printed.
```

Attention! `chol` command **does not** check the symmetry of the matrix!

LU decomposition with no pivoting

$$\left. \begin{array}{l} \mathbb{A}\mathbf{x} = \mathbf{b} \\ \mathbb{A} = \mathbb{L}\mathbb{U} \end{array} \right\} \longrightarrow \mathbb{L}\mathbb{U}\mathbf{x} = \mathbf{b} \longrightarrow \left\{ \begin{array}{l} \mathbb{L}\mathbf{y} = \mathbf{b} \\ \mathbb{U}\mathbf{x} = \mathbf{y} \end{array} \right.$$

Cholesky decomposition

$$\left. \begin{array}{l} \mathbb{A}\mathbf{x} = \mathbf{b} \\ \mathbb{A} = \mathbb{H}^T \mathbb{H} \end{array} \right\} \longrightarrow \mathbb{H}^T \mathbb{H}\mathbf{x} = \mathbf{b} \longrightarrow \left\{ \begin{array}{l} \mathbb{H}^T \mathbf{y} = \mathbf{b} \\ \mathbb{H}\mathbf{x} = \mathbf{y} \end{array} \right.$$

When you have to solve more systems $\mathbb{A}\mathbf{x} = \mathbf{b}$ with **varying** $\mathbf{b}$ and $\mathbb{A}$ s.p.d., compute just **once** the decomposition. Computational cost: $\mathcal{O}(\frac{2}{3}n^3)$ for LU decomposition, $\mathcal{O}(\frac{1}{3}n^3)$ for Cholesky decomposition.

Exercise 4.6. Consider the matrix

$$\mathbb{A} = \begin{bmatrix} 44 & 15 & 29 & 26 & 119 \\ 15 & 33 & 32 & 18 & 15 \\ 29 & 32 & 252 & 112 & 73 \\ 26 & 18 & 112 & 124 & 90 \\ 119 & 15 & 73 & 90 & 430 \end{bmatrix}$$

1. Verify that the Cholesky decomposition can be applied to matrix $\mathbb{A}$ (hint: use the command `eig`);
2. Write a Matlab function with signature `[H] = MyChol(A)` to implement the Cholesky decomposition of the matrix $\mathbb{A}$;
3. Compute the Cholesky decomposition of the matrix $\mathbb{A}$ by means of both the Matlab command `chol` and the function `MyChol` and compare the results;
4. Solve the linear system with $\mathbb{A}\mathbf{x} = \mathbf{b}$, $\mathbf{b} = [1, 1, 1, 1, 1]^T$

Solution Exercise 4.6.

ex_5_1.m

```
%% Exercise 4.4
% Write a function [InvA]=MyInv(A) that computes the inverse InvA of a
% generic square matrix A.
% Then use the function MyInv to compute the inverse of the matrix A in the
% previous exercise. Compare the result with the output provided by Matlab
% with command inv.

format shortG
A = [2 10 4 0;
     1 0 2 2;
     1 4 0 2;
     1 2 1 1];

InvA = MyInv(A)
InvA_mat=inv(A)
```

MyChol.m

```
function [H] = MyChol(A)
% order of the square matrix A
n = length(A(1,:));
H = zeros(n, n);

H(1, 1) = sqrt(A(1, 1));

for i = 2:n

    for j = 1: (i-1)
        H(i,j) = 1/H(j, j) * (A(i, j) - H(i, 1:j-1)*H(j, 1:j-1)');
    end

    H(i, i) = sqrt(A(i, i) - H(i, 1:i-1)*H(i, 1:i-1)');

end
H=H';

end
```

Exercise 4.7. Consider the system $\mathbb{A}\mathbf{x} = \mathbf{b}$ with

$$\mathbb{A} = \begin{bmatrix} 10 & 0 & 3 & 0 \\ 0 & 5 & 0 & -2 \\ 3 & 0 & 5 & 0 \\ 0 & -2 & 0 & 2 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 2 \\ 2 \\ 2 \\ 2 \end{bmatrix}.$$

- Compute the solution of the system. Use the Cholesky decomposition of matrix $\mathbb{A}$ if possible (check the hypotheses).
- Find the determinant of $\mathbb{A}$ without using the Matlab command `det`.

Solution Exercise 4.7.

hw_4_4.m

```
clc;clear;close all

A=[10 0 3 0; 0 5 0 -2; 3 0 5 0; 0 -2 0 2];
b=2*ones(4, 1);

if ( all(all(A'==A)) && all(eig(A)>0))
    fprintf("Cholesky HPS ok\n");
end
```

```
R=chol(A);  
y=R'\b;  
x=R\y  
  
disp(A\b)  
  
detA= prod(diag(R))^2  
disp(det(A))
```